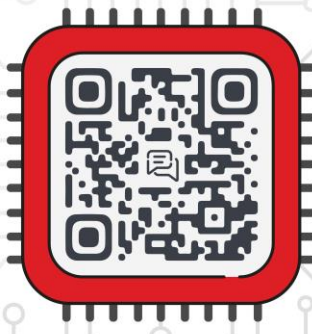




4

9

1080 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

طرق حل المشكلات باستخدام البرمجة التفرعية & التزامن



د. روان قرعوني

محتوى مجاني غير مخصص للبيع التجاري

RB Informatcs; 28/05/2025

برمجة تفرعية

طرق حل المشكلات باستخدام البرمجة التفرعية

هناك طرق نستخدمها لحل المشكلات البرمجية باستخدام البرمجة التفرعية:

1. الحسابات التفرعية الصريحة

2. فرق تسد

3. التجزئة

4. pipe line

إن أي مسألة تفرعية تخضع لـ 4 أنواع السابقة

1. الحسابات التفرعية الصريحة :

أجزاء المسألة لأجزاء لا يوجد تواصل فيما بينها.

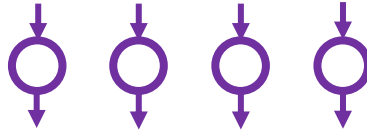
وتكون التجزئة مباشرة فيها، كل معالج يعلم مهمته ويقوم بها.

بحيث: تستخدم هذه الطريقة مع المسائل التي تتفرع بشكل كامل، أي لا يوجد ترابطات بين البيانات،

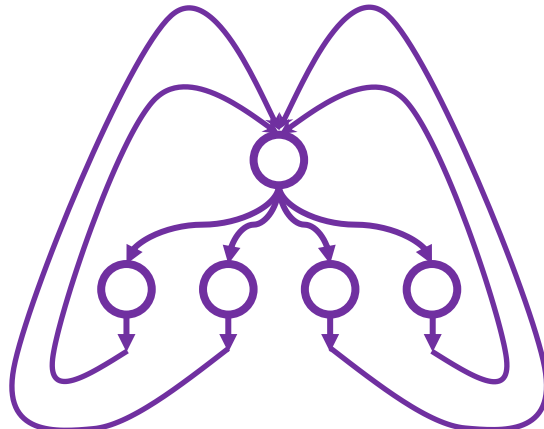
مثال: مصفوفة مربعة 4×4 ونريد تطبيق عملية على عناصرها، نقوم بتقسيم الـ Data، وتوزيع الأقسام على المعالجات

المتاحة بدون الحاجة لتواصل المعالجات مع بعضها، فلنفرض أن المعالجات المتاحة عددها أربعة، سيقوم كل معالج

بمعالجة 4 قيم من المصفوفة السابقة



كما يجب وجود مهمة مسؤولة عن توزيع الـ Data، وتجميعها بعد انتهاء المعالجات الأربعة من معالجة القيم .



2. التجزئة

تستخدم مع المسائل التي من الممكن أن تقسم فيها الـ data لأجزاء (كتل) وترسل كل كتلة إلى معالج خاص فيها تقسم هذه الطريقة الى:

- تجزئة معطيات (data) جمع 100 رقم، عوضاً عن جمعهم تسلسلياً $1+2+3+....$ اقوم بجمعهم لأجزاء، مثلاً كل 20 جزء لمعالج ومن ثم أجمع النتائج فيوجد data مختلفة لكل معالج مع نفس الوظيفة (الجمع)

■ تجزئة على مستوى الوظائف

يوجد معالج للضرب، معالج للقسمة فيوجد وظيفة مختلفة بين كل معالج والآخر ولكن نفس الـ data

■ ما الفرق بين طريقة التجزئة والطريقة السابقة " الحسابات التفرعية الصريحة "؟

تعتبر الطريقتين من طرق التفكير التفرعي حيث:

- في الأولى يكون لدينا مهمة واحدة تقوم بتوزيع الداتا على مجموعة معالجات، ثم تقوم بتجميع خرج هذه المعالجات
- أما في التجزئة، كل جزء من الداتا يطبق عليه نفس العملية، باستخدام أكثر من معالج، ثم يقوم أكثر من معالج بعملية التجميع.

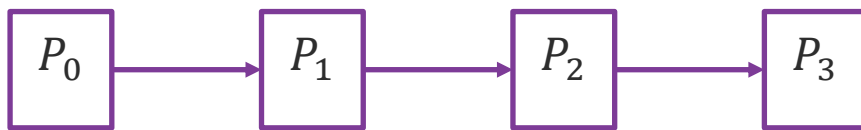
3. فرق تسد

تعتمد على مبدأ الشجرة، إذ يتم توزيع المعالجات على شكل شجرة يعني عوضاً عن العمل على 10 معالجات وأنا الأب أخذ نتائجهم يتم توزيعهم على شجرة وكل أب يأخذ نتيجة عمل المعالجات الأبناء.

مميزاتها : التعقيد لم يعد خطي (n) بل أصبح لوغاريتمي $O(\log n)$

4. pipeline

تكون للحسابات التي تحدث على أكثر من معالج حيث يكون لدينا تسلسل من العمليات



في المخطط السابق يبدأ التنفيذ من المعالج p_0 ثم يقوم بتمرير نتيجة التنفيذ إلى المعالج p_1 ، وبعد تمريرها يكون المعالج p_0 متاح لتنفيذ عمل آخر.

مثال :

- ليكن لدينا مصفوفة نريد أن نقوم بزيادة كل عنصر من عناصرها بـ x ، ثم نضربه بـ y ، ثم نقسمه على z وسنوزع العمل على 3 معالجات .
- نقوم بتمرير العنصر الأول في المصفوفة على المعالج الأول p_0 للقيام بالجمع، وبعد تمرير القيمة الناتجة للمعالج الثاني

- يقوم المعالج p_0 بتطبيق عملية الجمع على العنصر الثاني في المصفوفة .
- هكذا نضمن ان تكون الاستجابة أعلى، لأن جميع المعالجات تقوم بالمعالجة في الوقت نفسه .

هذه الطريقة لها ثلاث أنواع

▪ Pip(1)

يستخدم مع المسائل التي يكون لدينا فيها تطبيق نفس العمل على أكثر من دخل instance

- مثلاً لدينا

▪ (1) instance ستمر على المعالج P_0 ثم P_1 ثم P_2

وسيكون لدي لكل معالج فرضاً مهمة يقوم بها على المُدخل

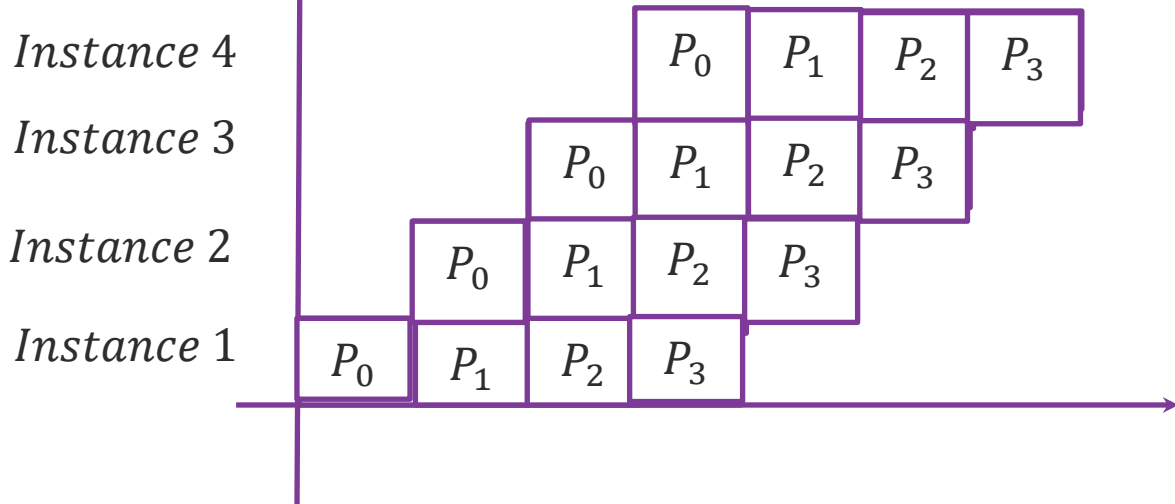
▪ لكن ماذا لو أتت فرضاً مصفوفة أو مُدخل (instance) جديد أثناء تلك العملية؟

عندها ستبدأ المصفوفة الجديدة وتدخل بالpipeline

المعالج لا ينتظر حتى انتهاء الpipeline كي يستقبل داتا جديدة

نلاحظ أن مبدأ عمله هو Single Instruction multiple Data .

لنقم برسم مخطط لتوضيح عملية معالجة ال instances .



لاحظ انه في المخطط السابق، بعد انتهاء المعالج P_0 من إجراء العمل المُوكَل إليه على ال instance الأولى، ينتقل مباشرة لمعالجة ال instance الثانية، وبهذا نقوم بزيادة الاستجابة من خلال تشغيل كل المعالجات في الوقت نفسه. لنفرض انه في المخطط السابق كان كل معالج يحتاج ثانية لإنهاء عمله على ال instance الواحدة، يكون زمن التنفيذ 7 ثواني، بينما في حالة انتظرنا كل instance لتنتهي قبل البدء بمعالجة التي تليها سيكون الزمن 16 ثانية.

STUDY NOW, BE PROUD LATER



Pip(2) ■

هذا النوع نفس مبدأ الأول ولكن تطبيقه على ال data مختلف

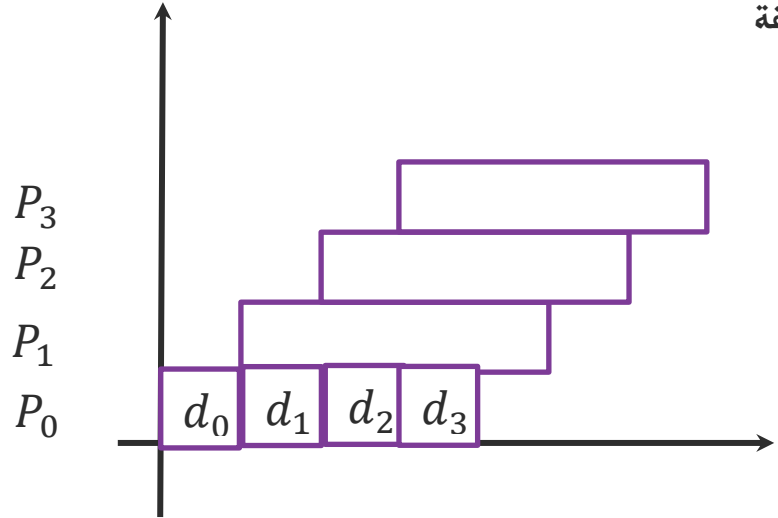
مثلاً: لدي معالجات $P_0, P_1, P_3, \dots$

يتعاملون مع data تمر عليهم وتنتقل من معالج لآخر

فرضاً لي data تدعى d_3, d_2, d_1, d_0

d_0 مرت بال P_0 ستنقل الى P_1 ثم على P_2

إذاً: تستخدم هذا النوع في حالة كان يجب مرور ال data ذاتها على جميع المعالجات كما في أمثلة التعديل على عناصر مصفوفة



بعد انتهاء المعالج p_0 من معالجة ال Data الـ d_0 يرسلها إلى المعالج p_1 ليقوم بعمله و ينتقل مباشرة لمعالجة d_1 .

Pip(3) ■

■ يستخدم عندما ليس هناك داعٍ لانتظار المعالج P_0 لينتهي من مهمته حتى انتقل الى P_1

فمن الممكن أن أستغل الوقت وأن أجعل باقي المعالجات تعمل

■ تعتمد على نوع المسألة .

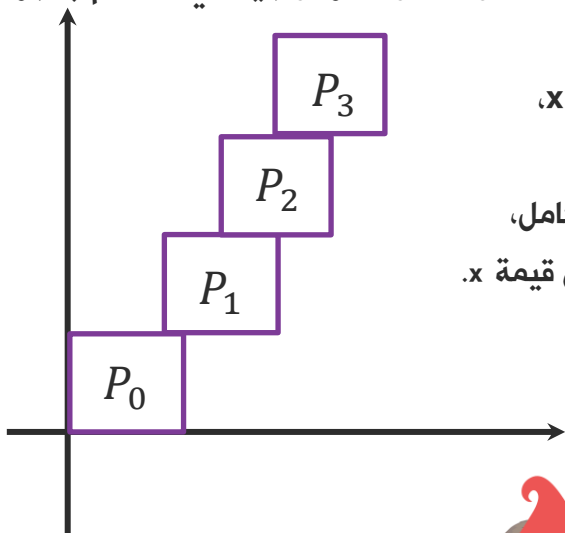
في بعض المسائل، قد لا تكون مضطر إلى انتظار المعالج السابق لتنفيذ كامل العمل المُوكل إليه، أي أنه قام بالجزء المطلوب ليتمكن المعالج التالي له من إجراء عمله بدون حدوث أخطاء،

مثال : ليكن لدينا المعالجين P_0, P_1 وكان المعالج P_1 يريد قيمة متحول x ,

قد ينتهي المعالج p_0 من تطبيق كامل عمله على قيمة المتحول x ,

في هذه الحالة يقوم المعالج P_1 ببدء التنفيذ قبل انتهاء المعالج p_0 بالكامل،

تحديداً في اللحظة التي ينتهي المعالج p_0 من إجراء كافة التعديلات على قيمة x .



مناسك وكن لأحلامك درع لا ينكس

. إذا هناك طريقتين للعمل فعلياً .

- المعالج الأول ينتهي ثم يرسل لمن يليه

-المعالج الثاني يبدأ عمله حتى قبل انتهاء المعالج الأول من العمل

التزامن

ما هو؟ التزامن بين عمليتين الأولى لا تعمل حتى تنتهي العملية الثانية

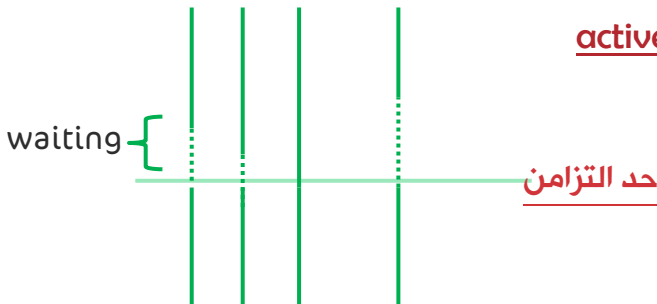
- العمل التفرعي يكون بتوزيع العمل على مجموعة معالجات، يقوم كل معالج بتنفيذ العمل الموكّل إليه، لكن لا بد من وجود التزامن في البرنامج بين هذه المعالجات، أي أنه لا يمكن وجود معالج يعمل وحده دون أي تواصل مع معالج آخر، من الأمثلة الممكنة وجود نقطة معينة، يتوجب على كل معالج الانتظار حتى تقوم جميع المعالجات الأخرى بإنهاء عملها، هذه النقطة تسمى بـ **Barrier** أو **حد التزامن**

مثال:

نريد إرسال رسالة لمجموعة **لا يصح** استخدام توابع gather & scatter قبل التأكد من أن جميع المهام المطلوبة انضمت للمجموعة، وإلا قد توجد مهام لن تستلم الرسالة.

في المخطط المجاور:

لدينا خط التنفيذ لأربعة معالجات، يقوم كل منها بالتنفيذ **مرحلة active** بعد الانتهاء تقوم بانتظار باقي المعالجات حتى تصل لـ **حد التزامن مرحلة waiting** ثم تتابع هذه المعالجات عملها.



أنواع التزامن

تقرّرنا البيئة وليس نحن وهي: 1-خطي 2- شجري 3- فراشة

1-التزامن الخطي

أسير على الـ Tasks واحدة تلو الأخرى لأضمن إنهاؤها ووصولها. وذلك عن طريق حلقة for من الأب لتعليمه الـ receive متى تنتهي عملية الاستقبال من الكل، ثم يقوم بحلقة for أخرى العملية الـ Send يُكتب إذاً بالأبناء الـ Send أولاً ثم ينتظر الـ receive

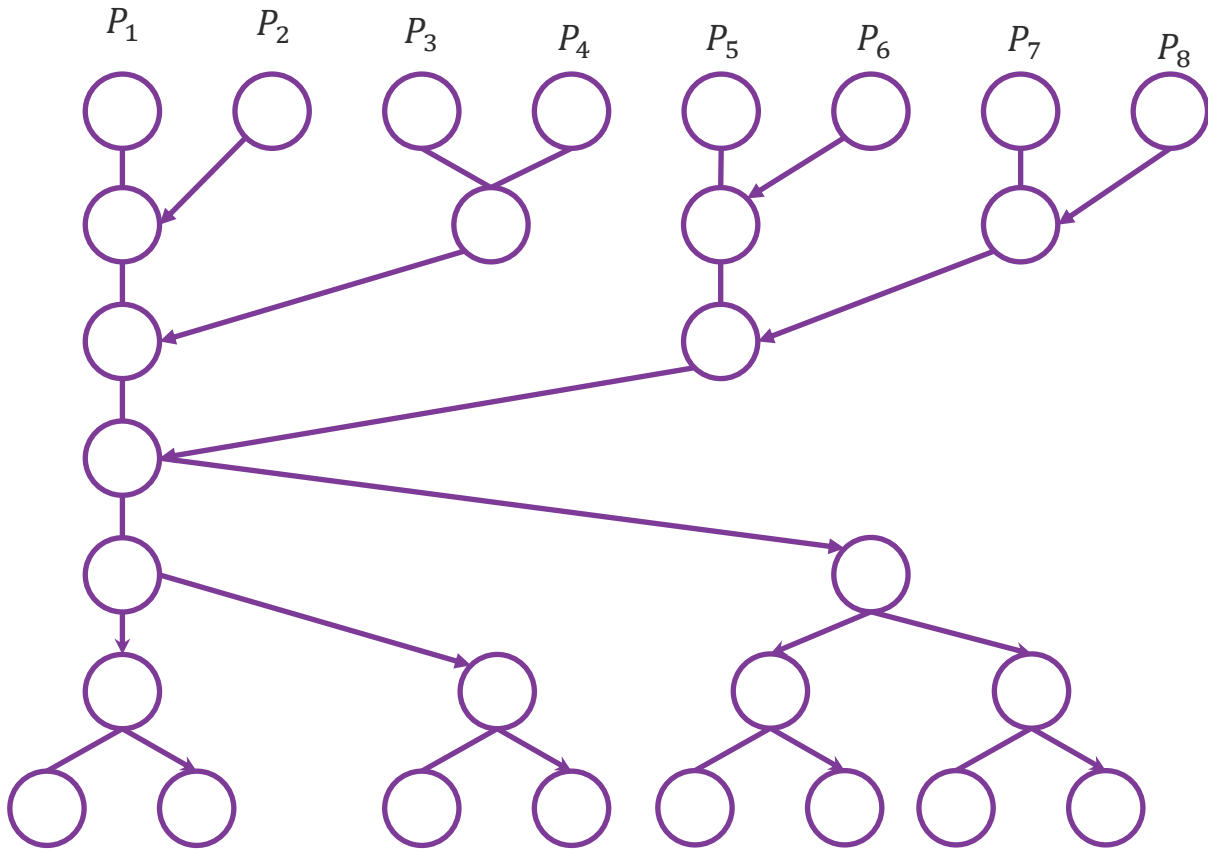
مشكلته: تعقيده (n) لأنه ينتظر كل مهمة من المهام حتى تقوم بالـ Send ومنها ينتظر كل التعليمات حتى يقوم بالـ receive بكل مهمة.

كل ابن يخبر الأب الخاص فيه عند الوصول لمهمة ما.

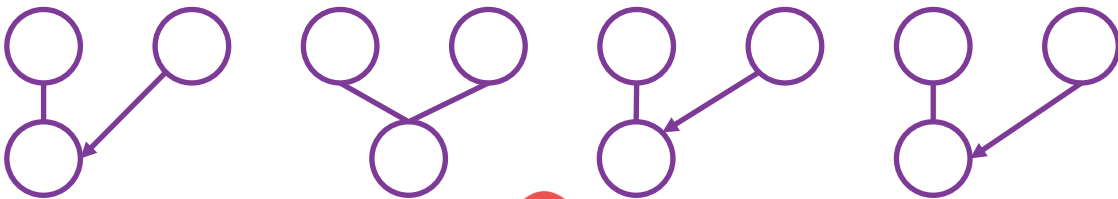
لو وصل اثنان في نفس اللحظة:

- بالحالة الخطية: كنا نحتاج لحلقات For وانتظارات طويلة.
- بالشجرة: كل عملية ترسل مباشرة لأبيها والأب يرسل لوالده فالتعقيد لوغاريتمي تعقيده $O(2\log(n))$ ، لأن العملية ذهاب وإياب (Send + retive)

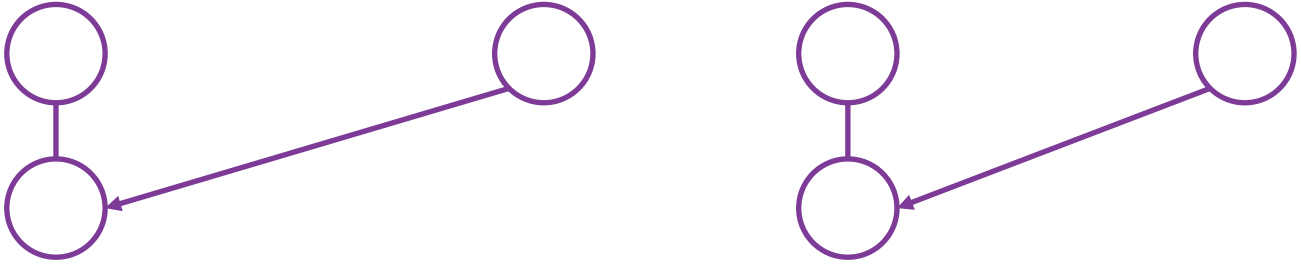
عوضا عن جعل المهمة master تنتظر جميع المهام ليكن لدينا 8 معالجات، من p_1 لـ p_8



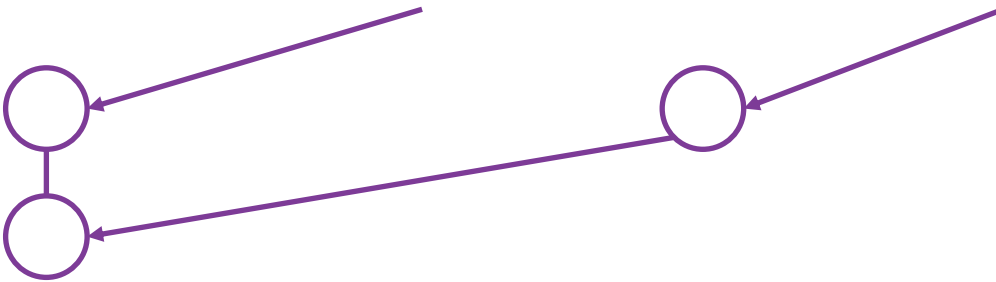
يقوم كل معالج ذو ترتيب زوجي بإخبار المعالج السابق له عند الوصول لحد التزامن في هذه الحالة تصبح جميع المعالجات ذات الترتيب الفردي تعرف حالتها وحالة المعالج الذي يليها



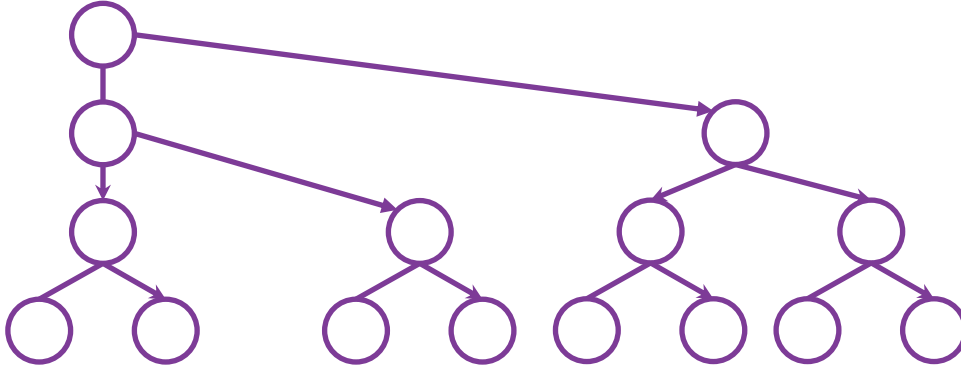
نقوم بتكرار العملية مع هذه المعالجات حيث يقوم المعالج الثالث بإخبار الأول ويقوم المعالج السابع بإخبار الخامس في هذه الحالة يكون المعالج الأول يعلم حالة المعالجات الأربعة الأولى، بينما يعلم المعالج الخامس بحالة المعالجات الأربعة الأخيرة.



فيقوم المعالج الخامس بإخبار الأول، فيصبح المعالج الأول يعلم حالة المعالجات الثمانية،



في هذه المرحلة تكون جميع المعالجات قد وصلت لحد التزامن، مما يسمح لنا بإرسال الرسالة وبترتيب معاكس للخطوات السابقة تكون عملية الإرسال



نلاحظ أنه في هذه الطريقة تكون عملية التواصل على التوازي .

3-تزامن الفراشة

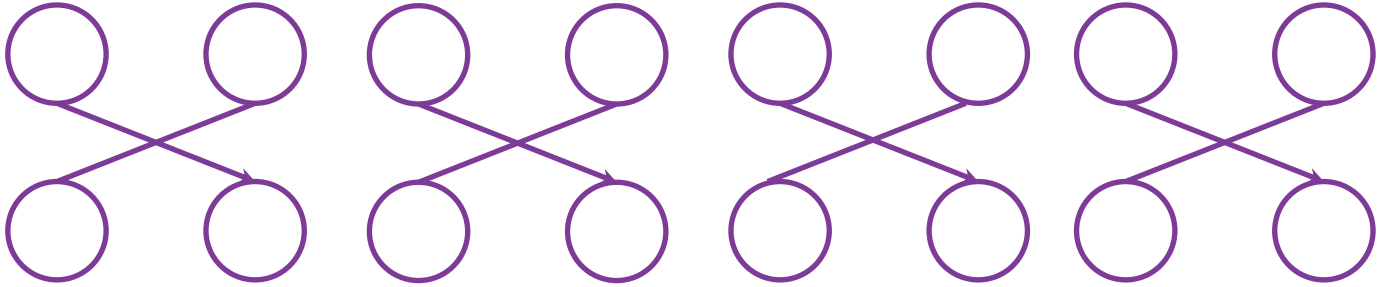
- كل مهمة تخبّر جارتها (أي تقوم بالإرسال والاستقبال في نفس الوقت)
- فتعقيدها لوغاريتمي ($O(\log(n))$) ولكن دون الضرب 2
- كل مهمة وصلت لرسالة أخيرة علمت أن كل المهام الأخرى وصلت

ليكن لدينا ثماني مهام مرتبة من p_1 لـ p_8



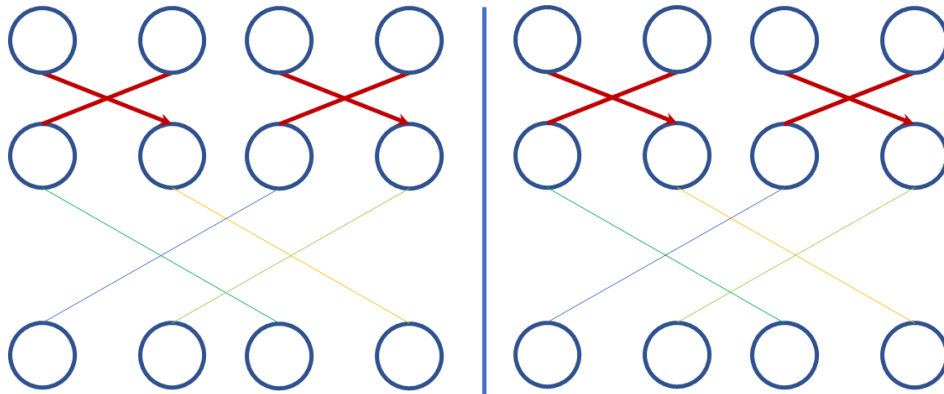
تقوم كل مهمة بإخبار المهمة المجاورة لها والاستلام منها، وذلك وفق:

- المهمة ذات الترتيب الفردي تقوم بالتواصل مع المهمة التالية لها
 - والمهمة ذات الترتيب الزوجي تقوم بالتواصل مع المهمة السابقة لها
- بهذا تكون كل مهمة لديها معرفة عن حالتها وحالة المهمة المجاورة



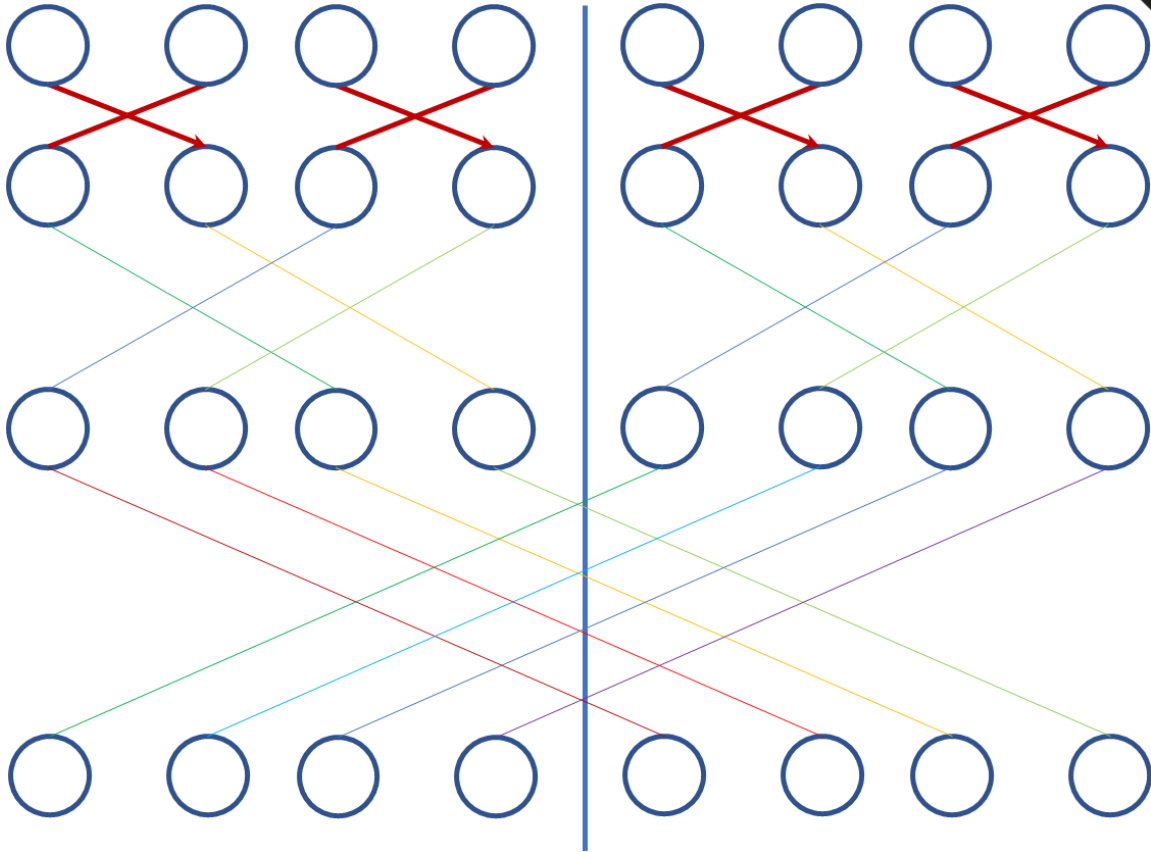
- لتبسيط الأمور نرسم خط وهمي يقسم المهام لنصفين،
- نهدف في هذه المرحلة لتأمين التواصل بين المهام في النصف الواحد أي تأمين التواصل بين المهام الأربعة الأولى، والتواصل بين المهام الأربعة الأخيرة .
- في كل نصف تقوم المهمة ذات الترتيب i بالتواصل مع المهمة ذات الترتيب $i+2$
- ما عدا:
- المهمة ما قبل الأخيرة التي تتواصل مع المهمة الأولى
- والمهمة الأخيرة التي تتواصل مع المهمة الثانية

في هذه الحالة تكون كل مهمة من المهام على علم بحالة المهام التي تقع معها في نفس النصف



- ننتقل للمرحلة الأخيرة وهي تأمين التواصل بين المهام جميعها من خلال تأمين التواصل بين كل مهمة من النصف الأول مع مقابلتها من النصف الثاني
- للتوضيح : نقوم بوصل المهمة الأولى من النصف الأول مع المهمة الأولى من النصف الثاني وهكذا

علينا السعي والله يتولى النتائج



- هكذا تكون كل مهمة من المهام الثمانية على علم بحالة المهام جميعها
- لنختبر صحة هذه الكلام على مهمة عشوائية ولتكن المهمة الخامسة
- المهمة الخامسة تعرف حالتها ومن المرحلة الأولى أصبحت على معرفة بحالة المهمة السادسة
- من المرحلة الثانية أصبحت على معرفة بحالة كل من المهام السابعة والثامنة (لأن المهمتين على علم بحالة بعضهما من المرحلة السابقة)
- ومن المرحلة الثالثة أصبحت على معرفة بحالة المهام جميعها بعد التواصل مع المهمة الأولى التي على علم بحالة جميع المهام في النصف الأول

هناك نوعان من التزامن.

1. Fully Synchronous

يجري على جميع المهام بالنظام، وتعتبر الأمثلة التي تناولناها سابقاً في هذا النوع .

2. Locally Synchronous

يتم تنفيذ التزامن على مجموعة محددة من العمليات داخل المهمة الواحدة - يستخدم في حال كانت المسألة لا تتطلب تزامن كل المام.



مبارك وصلنا الى نهاية المحاضرة